Kavach v1

Protocol specification. End-to-end encrypted location sharing and check-ins for circles of up to 100 accounts, through a server that stores only what it cannot open and deletes each record once every reader has it.

Kavach v1, protocol revision 4, formerly SealedStream SS5. Wire version 5. October 2026. The companion document "Kavach v1: Moyo implementation profile" holds the platform, server and retention choices of the first host.

How to read this
Sections 1 to 3 state the requirements, the terms and the cipher suite. Sections 4 to 12 are the protocol: accounts, recovery, membership, audience, epochs, records, delivery, durable state and reset. Sections 13 and 14 are the client contract and its results. Section 15 states what is and is not protected, 16 how conformance is shown, and 17 what we want a reviewer's opinion on. The appendix lists the scenario tests that gate release.
This document names no platform library, server product or retention value. Those belong to an implementation profile; the Moyo profile is the first.

Contents

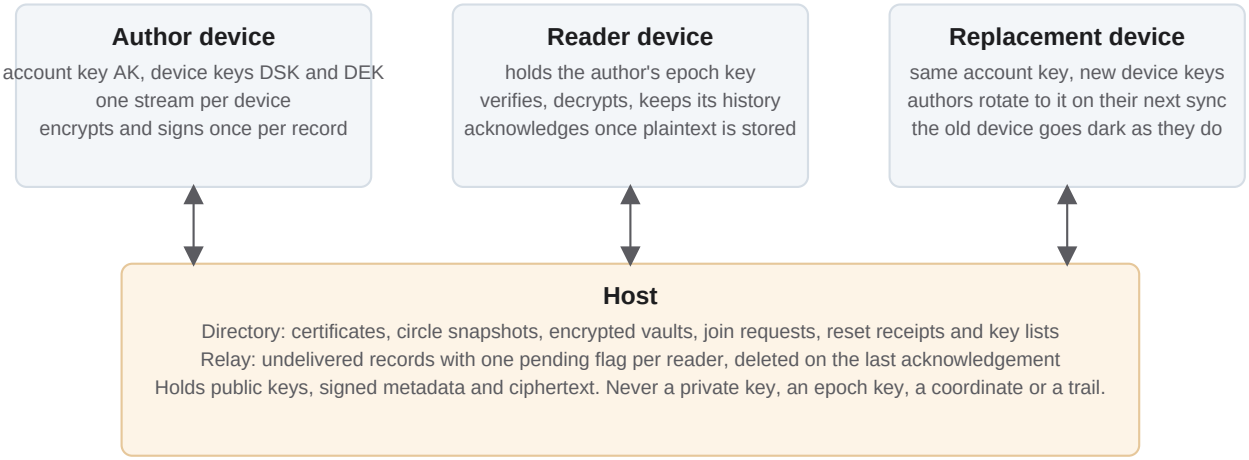
1. Scope and requirements
2. Terms
3. Encoding and cryptography
4. Account and device
5. Account vault and recovery
6. Circle membership and owner approval
7. Audience and reconciliation
8. Epoch manifest and addressed wraps
9. Records: locations and check-ins
10. Delivery and deletion
11. Durable state
12. Last-resort account reset and the reset keys
13. Client contract
14. States, errors and logs
15. Security properties and limits
16. Conformance
17. Questions for the reviewer

Appendix. Scenario tests that gate release

1. Scope and requirements

Kavach carries a person's location and check-ins to a small circle of people they chose, through a server that cannot read any of it. One account is one person. Each account has one active device, keeps its memberships when the device changes, and can own circles. The server authenticates accounts, stores opaque documents and ciphertext, forwards records, and deletes each record as soon as every recipient has acknowledged it.

This document is normative for Kavach v1. MUST and MUST NOT are requirements, SHOULD is a strong recommendation, MAY is optional. It is an implementation proposal, not an independently audited protocol. The machine-readable vectors described in section 16 travel with it.



Requirements

Requirement	How the protocol meets it
Membership belongs to the account	A circle lists stable account identities, never devices. The identity key survives a device replacement, so memberships and ownership do.
Owner continuity	The owner keeps the same account key and circle id on a new device. Replacing the owner's device never closes a circle or removes a member.
A new device keeps its circles, not its keys	Each device has its own encryption key. A replacement device recovers the account, publishes a new certificate, and receives new epoch keys as each author syncs. Membership continuity is guaranteed; an instant map is not. Replacement is also what cuts the old device off.
Owner approval only	A participant requests admission; the owner approves that account once. No invitation secret, no other-member signatures, no second ceremony.
History lives on devices	Devices keep what they received under the host's retention policy. A replacement device never imports old keys or history. The server keeps a record only until every recipient has it.
Recovery first, reset last	A new device recovers the account through a platform recovery provider or a saved recovery code. An explicit destructive reset exists for the case where both fail.
Check-ins are durable events	A check-in is a record kind with its own id. Location updates never overwrite it, and the host notifies recipients of it visibly.
Client-only SDK	The SDK owns cryptography and local security state. The host supplies authenticated login, a directory, an opaque relay, notifications and the interface. The SDK ships no server.

Fixed limits

A circle has 1 to 100 accounts including its owner. An author's effective audience is the union of its enabled circles plus itself, capped at 100 distinct accounts; an over-limit change is rejected or pauses sharing with `RECIPIENT_LIMIT`, never silently truncated. An account vault holds at most 100 circle entries. `accountName` is the host's immutable account identifier, 1 to 64 ASCII letters, digits, underscore or hyphen, case-sensitive.

Device generations, circle revisions, vault revisions and reset-key list versions are u64 starting at 1. Epochs are u32 starting at 1; record counters are u32 starting at 0. Nothing wraps: counter exhaustion rotates the epoch, epoch exhaustion starts a new device stream, revision or generation exhaustion is a terminal maintenance error. An author rotates on any known authorization change, on counter exhaustion, on clock reversal, and when its epoch is older than the profile's hygiene interval. Every document is at most 16,384 bytes; a plaintext payload is at most 4,096 bytes.

Out of scope

Multiple simultaneous devices per account, delegated owners, ownership transfer, reusable invites, peer key fulfilment, manifest pruning, ratchets, forward secrecy, post-compromise recovery, algorithm negotiation, guaranteed alert delivery and history migration to a new device. Section 15 names each where it affects a security claim.

2. Terms

Term	Meaning
Account	A person as the host knows them: an immutable `accountName` plus the Ed25519 account key `AK` generated on their first device. The identity id is derived from both.
Device	The one active device of an account. It holds a device seed from which its signing key `DSK` and its encryption key `DEK` derive, a random 16-byte `deviceId`, and a generation that rises by one at each replacement. Neither key derives from the account.
Certificate	The AccountDevice document (type 1): the account's current device generation and public keys, signed by `AK`. Its body hash is the `certificateHash` that manifests bind to.
Circle	A named group with one owner. Its membership is one owner-signed snapshot (type 2) listing stable identity ids.
Join request	Host state: an authenticated account asking to be admitted to a circle, or to leave it. It carries no signature. Only the owner's next snapshot changes membership.
Vault	The encrypted AccountVault (type 6): the account seed plus circle pins and sharing preferences, stored on the host under the recovery key. It holds no encryption key.
Stream	One author device's sequence of records; its id derives from the identity id and the device id.
Epoch	A span of an author's stream encrypted under one random key `SK`.
Manifest	The signed EpochManifest (type 3): the epoch's audience as (identity, certificate hash, wrap hash) rows plus a key commitment.
Wrap	A libsodium sealed box of `SK` to one recipient device's `DEK`: 80 bytes, unsigned, bound to the manifest by its hash.
Record	An EncryptedRecord (type 4): one location or one check-in, encrypted under `SK`, signed by `DSK`.
Audience	The identities an author encrypts to: itself plus every member of every circle it has joined and enabled, each at its current device.
Desired revision	A local u64 the author increments on every change that should alter its audience. An epoch prepared under an older value is obsolete.
Delivery window	The profile's configured time after the relay accepted a record during which it keeps it for readers that have not acknowledged it. After it, the record is deleted whether or not it was delivered.
Cursor	One u64 per reader per author stream, held by the relay: every record at or below it is acknowledged by that reader. The relay deletes a record once every reader the epoch addresses has a cursor past it.
Host	The server operator: authenticates accounts, stores opaque documents and ciphertext, forwards records, deletes them once delivered.
Reset receipt	An AccountResetReceipt (type 7) signed by a host reset key that a management-signed key list (type 8) currently allows.

3. Encoding and cryptography

All integers are big-endian and fixed width: u8, u16, u32, u64 unsigned, i32 two's complement. u64 values use exact integer arithmetic. `Fixed[n]` is exactly n bytes. `B(name)` is u8(byteLength) followed by the ASCII bytes. `||` is concatenation. The wire is binary. A signed structure has no optional fields, no padding, no trailing bytes. Implementations reject unsupported versions and types, invalid ranges and reserved bits, and consume every byte before any cryptography.

One suite

Function	Definition
`H(x)`	BLAKE2b, 32-byte output, unkeyed.
`C(label)`	ASCII("SealedStream/5/" + label) || 0x00. Labels are exact case-sensitive literals.
`D(seed, label)`	BLAKE2b with key = seed, message = `C(label)`, 32-byte output.
Signatures	Ed25519, detached, over `C(label) || body`. Not Ed25519ph, no intermediate hash.
Records, vaults	XChaCha20-Poly1305 with a 32-byte key, a fresh random 24-byte nonce and the 16-byte tag appended.
Key wraps	libsodium `crypto_box_seal`: 32-byte SK becomes an 80-byte box. The nonce inside is BLAKE2b with 24-byte output over the ephemeral and recipient public keys, as libsodium defines it.

The label prefix `SealedStream/5/`, the wire version byte 5 and the `SS5R-` recovery-code prefix keep the protocol's earlier name on purpose: Kavach v1 renames the SDK and the documents, not the wire, so every revision 2 vector still applies byte for byte.

Keys

Key	Origin and use
`accountSeed`	32 random bytes. AK is the Ed25519 keypair from this seed. Signs certificates and snapshots. Lives in the vault and the platform recovery item. Derives no encryption key.
`deviceSeed`	32 random bytes per installation, never backed up. DSK = Ed25519 from `D(deviceSeed, "device-sign")`; DEK = `crypto_box_seed_keypair(D(deviceSeed, "device-box"))`. The seed is never used directly as an X25519 scalar.
`deviceId`	16 random nonzero bytes per installation, new on every replacement.
`recoveryKey`	32 random bytes. Encrypts the vault. Carried by the platform recovery provider and the recovery code.
SK	32 random bytes per epoch. Never derived from any other key.

Identifiers

- `identityId` = `H(C("identity") || B(accountName) || AK)`. Stable across device replacements.
- `streamId` = `H(C("stream") || identityId || deviceId)`. Changes with the device.
- `circleId` and `resetId` are independent random 16-byte values.
- `recordId` = `H(C("record-id") || streamId || u32(epoch) || u32(counter))`.
- `bodyHash` = `H(C("doc-hash") || body)`. The `certificateHash` is the type 1 body hash.

Documents

Types 1 to 4, 7 and 8 have body = `u8(5) || u8(type) || fields` and wire = `body || signature[64]`. Type 6 is AEAD-protected and carries no signature. Type 5 was a signed join request in revision 2; it is retired, and a parser MUST reject it as `UNSUPPORTED_TYPE`.

Type	Document	Signer and label	Wire length
1	AccountDevice certificate	AK, `account-sig`	187 + name length (188 to 251)
2	CircleSnapshot	Owner AK, `snapshot-sig`	124 + 32 per member
3	EpochManifest	Author DSK, `epoch-sig`	208 + 96 per audience row
4	EncryptedRecord	Author DSK, `record-sig`	159 + plaintext length (159 to 4,255; the payload may be empty)
5	retired (was MembershipIntent)	none; rejected	none
6	AccountVault	none (AEAD under `recoveryKey`)	header 43 + name length, then 24 + 4 + ciphertext
7	AccountResetReceipt	Allowed reset key, `reset-sig`	171 + name length
8	ResetKeyList	Management key, `reset-keys-sig`	91 + 48 per key (0 to 8 keys)

Verification

Parse within bounds first, then verify the signature under the key the local trust state names, then apply continuity checks, then decrypt. Reject non-canonical Ed25519 signatures, invalid or small-order signing keys and X25519 keys that yield an all-zero shared secret. Only a type 1 certificate is verified under the AK it carries, and that establishes key possession only; every other verification key comes from an already pinned identity, or from the pinned management key for type 8. Two valid bodies at the same slot are `EQUIVOCATION`; an invalid signature is `BAD_SIGNATURE`.

4. Account and device

Type 1: AccountDevice certificate

```
body = u8(5) || u8(1) || B(accountName) || AK[32] || deviceGeneration:u64 || deviceId[16] ||
       DSK.public[32] || DEK.public[32]
wire = body || Ed25519(AK, C("account-sig") || body)
```

Generation is nonzero. `deviceId` is nonzero and new for that account. `identityId` is constant across all of an account's certificates; `streamId` changes with `deviceId`. A certificate changes only when the device changes: there is no same-device re-certification, so an author's current manifest, which binds the author's certificate hash, stays valid until the author itself is replaced.

First contact and pinning

At first sight of an account, the SDK pins (`accountName`, AK) through the host's trust path. In this profile that path is the authenticated host directory, so enrollment is **host-trusted**: a malicious directory could substitute a key at first contact. Fingerprint comparison between devices is supported for people who want to check, and does not change the wire. A later certificate MUST carry the same `accountName` and AK. A different AK for a pinned account is `ACCOUNT_KEY_CHANGED`: the SDK quarantines the new key until a reset receipt (section 12) is processed, and never closes circles or re-admits on an unsigned directory change.

Generation rules

Per known account the SDK persists the greatest accepted `deviceGeneration` and its `certificateHash`. A lower generation is `DEVICE_ROLLBACK`. The same generation with the same hash is idempotent; with a different valid body it is `EQUIVOCATION`. A higher self-contained certificate under the same AK is accepted; gaps are legal. A reused `deviceId` is `DEVICE_ID_REUSED`, and the host also enforces uniqueness. The host's directory accepts a new certificate only by compare-and-swap on the exact current hash and generation plus one. An accepted higher certificate increments the reader's `desiredRevision`, emits `deviceChanged`, and rotates: wraps go only to the new DEK.

Replacing the device

1. Recover the account: open the vault (section 5) with the recovery key from the platform provider or the saved recovery code. Verify the vault's account name and derive AK from the recovered seed before trusting anything in it.
2. Generate a fresh `deviceSeed`, `deviceId` and local store. Fetch the current certificate by the hash the vault pins and the snapshots of the vault's circles; verify them against the recovered pins.
3. Prepare the new certificate at generation plus one and a vault at its next revision. Persist both as exact bytes (T2), then call `activateDevice` with the expected current certificate hash and vault revision. A lost response is retried with identical bytes, never regenerated.
4. On commit the host moves the account's current-device pointer, fences the old device from all writes, and hints every circle the account belongs to. The new device starts epoch 1, counter 0 in its new stream once its audience is resolved and its first manifest is published.
5. Each author rotates to the new DEK on its next sync. Until an author rotates, every record it sends is sealed to the old certificate and never reaches the new device's subscription; the new device sees that the author's current manifest has no row for it, and the host's interface shows that person as waiting rather than a blank map. Nothing waits on the circle owner.

Activation changes only the current-device pointer, the vault checkpoint and the transport fence. Nothing from the old device is copied: no outbox, no SK, no counters, no `deviceSeed`, no history. A missing or corrupt local store is handled by the same procedure under the preserved account, never by account reset.

The retired device

A device that learns of a newer certificate for its own account enters `retired`: it stops all author and owner writes, cancels pending uploads, erases its device seed and working secrets best-effort, and keeps whatever local history the profile keeps. It MUST NOT delete the platform recovery item. Its DEK is the only key that opens the wraps addressed to it, so once every author has rotated it can read nothing new; that is the revocation a replacement provides, and it is complete only when the last author has synced. Replacement is not recovery from compromise of `accountSeed`: whoever holds that seed can authorize a device of their own, and only a reset ends that.

5. Account vault and recovery

Type 6: encrypted AccountVault

```
header = u8(5) || u8(6) || B(accountName) || AK[32] || vaultRevision:u64
AAD    = C("vault-aad") || header
wire   = header || nonce[24] || ciphertextLength:u32 || ciphertextAndTag

plaintext = accountSeed[32] || lastDeviceGeneration:u64 || lastCertificateHash[32] ||
            circleCount:u16 || entries
entry     = circleId[16] || ownerIdentityId[32] || sharingEnabled:u8
```

Encrypt under `recoveryKey`. Revision is at least 1; ciphertext length is 90 to 4,990 bytes. Entries sort by `circleId` and are unique, count 0 to 100; `sharingEnabled` is 0 or 1. An entry pins a circle and the owner the user agreed to; whether the account is admitted or still waiting is read from the circle's current snapshot, never stored here. The vault holds no location, no SK, no `deviceSeed`, no DEK, no counters and no history: holding it with its recovery key authorises a device, and with it future access, but decrypts no history. After decryption, verify the account name and that AK derives from the recovered seed before trusting the entries.

Recovery order

1. **Platform recovery provider.** The platform's synced secret store hands back the `recoveryKey` for this account on a new device signed into the same platform account. No code, no old device.
2. **Saved recovery code.** `"SS5R-" || base64url(recoveryKey)`, 48 characters, available on demand after local authentication. It opens the latest vault the server returns and has no freshness anchor.
3. **Explicit destructive reset** (section 12), only when the user confirms, never automatically.

Every path yields the same account key, circles and preferences, and fresh device keys. When both recovery paths fail the SDK returns `ACCOUNT_RECOVERY_REQUIRED` and preserves the circle state it cannot use. It never manufactures replacement identity keys. Enrollment never blocks on the recovery code: the SDK saves the key through the provider and returns; the code is offered later as the way to complete recovery when the provider cannot.

Recovery provider contract

The SDK takes a `RecoveryProvider` with `load(accountName, identityId)`, `save(accountName, identityId, recoveryKey)`, `status()` and `eraseLocalAccess()`. Results carry a status (`available`, `unavailable`, `locked`, `unsupported`, `error`), a stable reason and a `backupState` (`unknown`, `pending`, `unavailable`). A successful local write is never reported as a completed cloud backup. The host authenticates the account before the provider releases a secret; the SDK verifies the recovered vault before activation. Logout or retirement erases local access without deleting the shared item. Provider failure never strands enrollment and never triggers a reset.

Vault updates

Vault changes are control-plane work and rare: a join request adds an entry before the request is sent, a leave removes it, enabling or disabling sharing updates the preference, and device activation records the new generation and certificate hash. Observed snapshots never touch the vault, and owner edits publish the snapshot alone. The SDK persists the exact encrypted bytes, increments the revision and publishes by compare-and-swap on the previous revision and hash; on conflict it fetches and decrypts the current vault and recomputes. A recovering device fetches each pinned circle's current snapshot, verifies it under the pinned owner identity, and learns from it whether the account is admitted or still waiting. A server can withhold or replay an old vault; a fresh device cannot detect that, and what it loses is a circle pin or a preference, never a key or a membership the owner granted. Section 15 states that limit.

6. Circle membership and owner approval

Type 2: CircleSnapshot

```
body = u8(5) || u8(2) || circleId[16] || ownerIdentityId[32] || revision:u64 ||
       memberCount:u16 || memberIdentityId[32] x memberCount
wire = body || Ed25519(ownerAK, C("snapshot-sig") || body)
```

Members sort strictly by identity id, are unique, count 1 to 100, and include the owner exactly once. Resolving members MUST reject two different account keys for one account name. Creation is revision 1 with the owner as sole member; the SDK pins (`circleId`, `ownerIdentityId`) at creation or at a user-confirmed join, never from a server-supplied list. Per circle the SDK persists the highest accepted revision and body hash. A higher self-contained snapshot may skip intermediates; a lower revision is `ROLLBACK`; the same revision with the same body is `DUPLICATE`, with a different valid body `EQUIVOCATION`, which quarantines the circle. A device change neither resets the revision nor creates a snapshot. A restored owner fetches the current snapshot and signs revision plus one; the host's compare-and-swap prevents overwriting a newer one.

Join and leave requests

A request to join or leave is host state, not a document. The host already authenticates the account that makes it, and the owner's trust in that account's key comes from the directory pin, exactly as for every other account. A signed request would prove only what the host's authentication already establishes, at the cost of a document type, an expiry policy and a replay list, so revision 3 removed it. Requests are transient and never enter a membership chain: only the owner's next snapshot changes membership.

Action	Rule
Request to join	The requester stores the pinned circle (`circleId`, `ownerIdentityId`) and its sharing preference in its vault, then calls `requestJoin` on the host, authenticated as its account. The host records the pending request and notifies the owner visibly, naming the requester and nothing else. A link or short code the host hands the requester is a routing hint, never membership authority.
Owner approves	The owner reads the pending request, resolves and pins the requester's (`accountName`, `AK`) through the directory, builds the successor snapshot from trusted local state plus this one change, commits it (T8) and publishes it by compare-and-swap. The host clears the request when the snapshot that admits the account is published. Approval of an admitted account is idempotent.
Start sharing	`requestJoin` defaults to sharing enabled on approval, because that is the user's explicit request; the host may pass false.
Replace a device	The snapshot does not change. Verify the member's new certificate under the pinned account key and rotate to its new device key.
Leave	`leave` disables local sending, removes the vault entry and stops opening that circle's authors immediately, then files a leave request so the owner removes the membership.
Remove	The owner's explicit action: publish the next snapshot without that identity. Each author pauses and rotates when it learns the change.
Close	Only explicit `closeCircle` prepares an owner-only snapshot and closes the circle locally. Device loss, replacement or absence never closes a circle. The owner cannot leave without closing.

A pending request expires on the host after a profile-chosen time; a rejected or expired request cannot block unrelated changes. A fabricated request shown by a malicious host yields nothing a first-contact key substitution could not: the owner admits the account the request names, under the key the directory serves, and section 15 counts both under host-trusted enrollment.

7. Audience and reconciliation

An **eligible circle** is one the account has locally joined and enabled, whose snapshot contains this account, whose owner pin is verified, and which is neither closed nor quarantined. The SDK keeps using the last verified snapshot while reporting its age; a malicious server may withhold a newer removal, and section 15 says so.

The **desired audience** is the author's own identity plus every member of every eligible circle, deduplicated. Each identity resolves to its greatest verified current certificate; an audience entry is (`identityId`, `certificateHash`, `DEK.public`). Exactly one current device per account receives a wrap. An ambiguous name-to-key mapping is rejected; a changed AK is a recovery or security error, never silent substitution. A self-only audience is valid.

Desired revision. The author persists `desiredRevision` and increments it for a newly observed relevant snapshot revision, an accepted recipient device change, its own device activation, a join, leave, enable or disable, counter exhaustion or hygiene rotation. It increments even when a newer snapshot's final member set is unchanged, so a missed remove-then-rejoin still rotates. The same transaction (T3) invalidates the active epoch, burns allocated counters, cancels not-yet-handed-off ciphertext sealed under the obsolete revision, and sets `paused(rotation)` before any callback runs. Bytes already handed to the network cannot be recalled, which is why section 11 serialises the handoff with this transaction.

If any required identity, snapshot or current-device proof is missing, return `METADATA_REQUIRED`, pause enqueue and publication, and never guess, omit recipients or reuse a known-obsolete key. A valid conflicting signature quarantines the affected state; invalid external bytes alone never replace valid cached state.

Sync cadence. The SDK synchronises on connection, resume, host metadata hints and at a profile-defined interval while online; the host forwards background wake-ups. After a reconnect or any transport uncertainty, synchronise before draining the outbox. On accepting a new recipient certificate, emit `deviceChanged` and rotate immediately; after rotation emit `needsLocation` so the host supplies a fresh sample rather than resending an old one.

8. Epoch manifest and addressed wraps

Type 3: EpochManifest

```
body = u8(5) || u8(3) || streamId[32] || authorIdentityId[32] || authorCertificateHash[32] ||
       epoch:u32 || createdAt:u64
       || keyCommitment[32] || audienceCount:u16 || rows
row  = recipientIdentityId[32] || recipientCertificateHash[32] || wrapHash[32]
wire = body || Ed25519(DSK, C("epoch-sig") || body)

keyCommitment = H(C("epoch-key") || streamId || u32(epoch) || SK)
wrap          = crypto_box_seal(SK, recipient DEK.public)        -- exactly 80 bytes
wrapHash      = H(C("wrap") || wrap)
```

Rows sort strictly by identity id, are unique, count 1 to 100, and include the author exactly once. A reader fetches the author's certificate by its hash, verifies it under the pinned AK, and requires that its derived identity id and stream id match the manifest and that it is the author's current certificate before using the epoch for a live location. The reader's own row must name its current certificate hash. Before storing SK, verify the manifest signature, the certificate bindings, the own row, the wrap hash, sealed-box authentication and the key commitment. Signing is required because every member knows SK.

Publication

Step	Rule
Prepare	Resolve the full audience; choose `highestCommittedEpoch + 1`; generate SK, one wrap per row; sign once. Persist the exact manifest, all wraps, SK and the captured `desiredRevision` in one local transaction (T4) before any network call. At most one unresolved epoch per stream.
Publish	`publishEpoch` compares the host's current epoch and hash, installs the bundle atomically and raises the stream's epoch fence. A timeout is retried with identical bytes; a lost response is resolved by lookup: matching bytes mean committed, a different valid body at that epoch is `EQUIVOCATION`. Never regenerate a signature, nonce or key for a prepared slot.
Promote	After commit, persist `highestCommittedEpoch`. Promote to active only if the captured `desiredRevision` and the exact certificate-bound audience still match current state; promotion stores the active SK and `nextCounter = 0` in one transaction (T5).
Obsolete	Otherwise the epoch is committed but obsolete: never seal with it, prepare the next one.
Exhausted	At counter 2^32 - 1, on clock reversal, or when the epoch is older than the hygiene interval, the next enqueue returns `ROTATION_REQUIRED` and a new epoch is prepared.

Retention of epoch material

An epoch's manifest, wraps and author certificate stay on the host while an undelivered record references the epoch or the epoch is the author's current one; when neither holds the host deletes them. A reader keeps an opened SK until every record of that epoch it holds has been decrypted into local history or has expired. Generation and epoch checkpoints are persisted separately from key deletion. A replacement device never fetches old-device wraps or keys. An unexplained higher committed epoch for the local stream means local state was lost; the device replaces its stream under the same account rather than guessing counters. A self-wrap recovers SK, never a counter.

9. Records: locations and check-ins

Type 4: EncryptedRecord

```
header = u8(5) || u8(4) || streamId[32] || epoch:u32 || counter:u32 || sentAt:u64 ||
         flags:u8     -- 51 bytes
flags  : bit 0 reserved, MUST be zero; bit 1 = kind (0 location, 1 check-in); bits 2-7 zero
AAD    = C("record-aad") || header
body   = header || nonce[24] || ciphertextLength:u32 || ciphertextAndTag
wire   = body || Ed25519(DSK, C("record-sig") || body)
```

Bit 0 carried an urgency flag in an earlier revision; this revision reserves it, so existing vectors remain valid and a check-in's kind alone decides how the host notifies. One global counter spans both kinds in an epoch. `sentAt` is the public send time in Unix milliseconds, not an authoritative capture time. Encrypt with the active SK and a fresh nonce; ciphertext length is plaintext length plus 16, between 16 and 4,112. The signature covers the entire body; the AAD covers the header. A retry resends the persisted bytes and never changes

nonce, counter or `sentAt`.

Verification. Resolve the author's current certificate and the manifest for (`streamId`, `epoch`); verify stream and certificate bindings, then the `DSK` signature, then decrypt. Zero epoch, reserved flags, out-of-range lengths and trailing bytes are invalid. A known superseded author device cannot supply a new live pin (`SUPERSEDED_DEVICE`); its previously accepted records may still be verified under the archived certificate and manifest and stored as history.

Location codec

```
plaintext = payloadVersion:u8 = 1 || latitudeE7:i32 || longitudeE7:i32 || accuracyMm:u32 ||
            capturedAt:u64 || status:u8   -- 22 bytes
```

Latitude in -900,000,000 to 900,000,000; longitude in -1,800,000,000 to 1,800,000,000; accuracy 0 to 100,000,000 millimetres. `capturedAt` is the encrypted capture time, shown to the reader as fix age. Status 0 normal, 1 stale, 2 unavailable; unavailable requires all-zero coordinates and accuracy. Unknown versions or statuses and trailing bytes are `BAD_PAYLOAD`. The codec carries no names, addresses or device identifiers. The byte API accepts any opaque payload up to 4,096 bytes.

Check-ins

A check-in is a record with kind 1. Its `recordId` is its stable event id. It may carry the location codec or an application payload. Each accepted check-in is durably queued and retried until server acceptance, explicit security cancellation or configured expiry; a later location never replaces it, in the outbox, on the relay or on the reader. The host notifies each recipient of a check-in visibly, with no record content; the protocol does not promise that every person saw it.

10. Delivery and deletion

Publishing

`enqueueLocation` and `enqueueCheckIn` run inside one write transaction (T6): require a usable active epoch whose `desiredRevision` equals current state, enforce the profile's queue limits, encrypt and sign once, insert the complete wire into the outbox, write the author's own history entry, and increment `nextCounter`. `queued(recordId)` is returned only after commit. Accepted queued records are never coalesced or replaced; `QUEUE_FULL` is explicit.

A single serialised sender drains the outbox in (epoch, counter) order with one record in flight per stream: the next handoff waits until the previous outcome is durable, so accepted (epoch, counter) pairs reach the relay in strictly increasing order and the relay's append checkpoint below stays small. For each record it opens a write transaction, re-checks device status, epoch, `desiredRevision` and expiry, marks the row in flight, **initiates the handoff to the transport inside that transaction**, and commits; only awaiting the response happens outside (T6h). Because T3 needs the same lock, no authorization change can commit between the check and the handoff. A timeout retries the same bytes while still eligible. A known change cancels pending uploads sealed for the obsolete audience and emits `cancelled(recordId, reason)`. Old keys are never re-wrapped to newly admitted accounts to deliver a backlog.

Relay contract: deliver, then delete

The server keeps a record only until every recipient has it. The relay is a per-author stream with one cumulative cursor per reader, not an archive and not a per-record ledger. It is trusted for delivery and expiry bookkeeping, never for confidentiality or authenticity: it can lose or expire a record, which is an availability failure, and it can never forge one.

1. **Append.** `appendRecord(recordId, wire)` verifies the caller's authorization and the current device and epoch fences, stores the immutable ciphertext once, and assigns it the stream's next `deliverySeq` (a u64, strictly increasing, never reused) and an acceptance time that never decreases, in one transaction. It is idempotent by `recordId` and exact bytes: a retry returns the original sequence; a different body for a used id is `EQUIVOCATION`; a retry after deletion is answered by the append checkpoint below and is never reinserted. A new record can only be appended for an epoch whose manifest the relay already holds and that is the author's highest published one; after verifying the author device, the relay consults the append checkpoint below before applying that epoch fence, so a retry of the previous epoch's last record gets its original outcome rather than a rejection, and returning that outcome appends nothing.
2. **Readers.** A reader's subscription to a stream is bound to (`streamId`, `recipientIdentityId`, `recipientCertificateHash`). The host creates it at the stream's current tail when the reader is admitted, when the reader activates a device, and for every member when an author opens a new stream, and it holds one `ackCursor` starting there. A replacement device gets a new subscription at its activation; it never inherits the old device's cursor, and records sealed to the old certificate are never re-addressed to it.
3. **Read.** `readRecords(streamId, afterSeq, limit)` answers from one consistent view of the stream and the caller's subscription. It scans strictly above `scanStart = max(afterSeq, committedCursor, expiredThrough)` and rejects an `afterSeq` beyond the tail. It returns, in sequence order, every retained record in `(scanStart, nextSeq]` whose epoch addresses this subscription's certificate, together with `nextSeq`, the stream's `tail`, its `expiredThrough` watermark and the subscription's committed cursor. `nextSeq` is the last sequence fully examined, never an assumed tail: when the limit is reached the page stops before the first eligible record it could not include, and an empty page may still advance through an examined interval that held nothing eligible. It writes nothing; the reader's scan position is local state. A sequence inside the interval that is not returned was not addressed to this subscription, was deleted after every required reader acknowledged it, or lies within the expired prefix; the reader acknowledges through `nextSeq` without telling these apart, and only once every returned record is durably processed and every earlier blocker it knows of is resolved or expired. A page limit or a later page never clears an earlier blocker. The relay is trusted for delivery: withholding a record is an availability failure it could commit through expiry in any case, and it is never a forgery.
4. **Acknowledge.** `ackThrough(streamId, cursor)` rejects a cursor above the tail with `CURSOR_OUT_OF_RANGE` and otherwise sets the subscription's cursor to max(stored, requested), authenticated to the exact reader device and subscription. An equal or older cursor is a successful no-op with no write. Success is reported only after durable commit. There is no per-record acknowledgement and no exception list.
5. **Completion and immediate deletion.** A record's required readers are the rows of its epoch's manifest other than the author. When a cursor advances, the relay deletes every record whose sequence is now at or below the cursor of every required reader, in the same transaction as the cursor change. The minimum is taken over the epoch's audience, never over every subscriber the stream ever had. A self-only epoch's records complete on append. No copy survives on the server: record bodies are excluded from the host's logs, analytics, backups and dead-letter queues, which the deletion promise covers. Physical erasure of replicas is operational policy the client cannot prove.
6. **Expiry.** One delivery window applies to every record of a stream. A record expires at its acceptance time plus the window; because acceptance times never decrease, expiry advances as a prefix. The relay keeps one monotonic `expiredThrough` watermark per stream, deletes the ciphertext of the expired prefix, and returns the watermark on every read so a reader can cross the prefix without asking for records that no longer exist. It never expires a newer record because an older one expired. The author's

own history is then the only copy on the author's side; readers that already stored a record keep it. Expiry is an availability fact, never a receipt.

7. **Append checkpoint.** The relay keeps per stream one high-water (epoch, counter) pair and the outcome of the last append (`recordId`, body hash, `deliverySeq`, acceptance time), updated atomically with each append. Within the profile's `appendRetryWindowMs`, a retry matching the last outcome returns it even after the ciphertext was deleted; a different body at that slot is `EQUIVOCATION`. An append at or below the high-water pair with no matching evidence is `STALE_APPEND`: nothing is reinserted and no success is invented. The client records that record as `acceptanceUnknown`, keeps its own history entry, stops retrying it and, for a location, asks for a fresh sample; it never re-creates a check-in on its own, which could duplicate the event. Appends for a retired device stream are refused by the certificate fence and never recreate deleted stream state.
8. **Epoch material.** A manifest, its wraps and the author's certificate are retained while the epoch is current or any retained record references it, and deleted when neither holds; a certificate shared by several epochs or by trust state stays while any of them still needs it. The expiry watermark is one less than the sequence of the oldest retained record whose deadline has not passed, or the tail when nothing is retained, so records already deleted by acknowledgement need no timestamps of their own.
9. Sequences, cursors and watermarks are transport state, never authorization. A missing record is `NOT_FOUND`, not proof that it never existed.
10. On a check-in append the host notifies each recipient visibly; on a location append it wakes the recipient silently. Neither carries record content.

Receiving

Only accept authors currently admitted in a locally joined circle, or self. `receive(wire)` resolves the author's certificate and the manifest for (`streamId`, `epoch`), fetching the manifest and the reader's wrap through `getEpoch` when they are not local, verifies the complete record, and in one transaction (T7) inserts it by `recordId` into local history, records a disposition, and updates the live pin only if the record is kind 0 and its (generation, epoch, counter) is greater than the pin's. Replays are harmless upserts; a different valid body for a retained `recordId` is `EQUIVOCATION`. There is no application-level acknowledgement: relay acknowledgement is internal to the SDK.

Acknowledgement rule. Per author stream the SDK keeps a local acknowledged prefix and advances it only across a contiguous run of sequences in which every record addressed to this device has been durably stored, or that the relay has declared expired. It sends at most one `ackThrough` per stream per processed batch, after T7 commits. A record it cannot open yet because it is offline and cannot fetch the epoch is `keyPending`: it is not stored, the prefix holds at it, and records beyond it are still read, verified and stored, so the next acknowledgement covers them. The relay returns only records addressed to this subscription; a returned record whose verified manifest nevertheless has no row for this device is `invalid(NOT_AUTHORIZED)` and does not hold the prefix, because this device is not among its required readers. Acknowledging a `keyPending` record early would let the relay delete the only copy of the key material, and a later rotation cannot recover an earlier epoch's random key. Crossing an expired prefix is queue progress, never a receipt.

Records arrive out of counter order and that is normal: counter 6 after counter 7 is inserted into the trail and cannot move the live pin backward. The SDK never waits for contiguous counters. Records from a superseded device populate history when the receiver holds the original key; they cannot overwrite the current device's pin. Removing a member clears that member's live pin and keys; it does not erase history already received. No peer key requests exist: the only sources of a key are the author's manifest and the author's next rotation.

`receive` **result**	**Meaning**
`updated`	New record stored; the live pin moved (kind 0) or a check-in was added.
`history`	New record stored in the trail; the pin did not move because a newer location is already shown.
`duplicate`	Same bytes already stored. Acknowledge, do not reinsert.
`keyPending`	Not stored; the acknowledged prefix holds; read again after the next sync.
`expired`	Inside the relay's expired prefix, or past `sentAt + 5 minutes` of future skew (`CLOCK_SKEW`).
`invalid(reason)`	`BAD_SIGNATURE`, `DECRYPT_FAILED`, `SUPERSEDED_DEVICE`, `NOT_AUTHORIZED`, `EQUIVOCATION` or a parse error. Nothing is updated.

`latest()` is a view over verified kind 0 records with capture time and age; a stale pin is never presented as live. A replacement device subscribes at its activation boundary and receives only records appended after it; its map fills as authors rotate, person by person.

11. Durable state

This section states what the local store must guarantee. Which database or secret store provides it is a profile decision.

Requirements

- **Secrets.** `accountSeed`, `deviceSeed`, `recoveryKey` and the store's own key live in the platform's protected secret store, device-only. Only the `recoveryKey` is ever placed in a synced platform item. The local store, device seed, counters and outbox are excluded from automatic device backup.
- **Encryption at rest.** Every row of the local store, including plaintext history, is encrypted at rest under a per-installation key held in the secret store. No plaintext secret appears in a row key, an index or a log.
- **Binding.** A `meta` row binds schema version, a random store instance id, `identityId` and the current `certificateHash`. A mismatch between the secret store and the local store pauses with `RECOVERY_REQUIRED` and is resolved by device replacement, never by inventing counters.
- **One writer.** All writes from every execution context of the app run under one mutual-exclusion primitive that spans processes. Reads that only inspect state may run outside it.
- **The handoff.** The lock protects only what happens inside it, so the one network operation whose ordering matters, the handoff of an outbound record, is initiated inside it (T6h). Every other network call runs outside, and its result is recorded in a new transaction that first revalidates `desiredRevision`, the certificate generation and the epoch; a context whose revalidation fails discards its result.
- **Bounded waiting.** A writer that cannot acquire the lock returns `RETRYABLE` without allocating counters or keys.

Transaction boundaries

Tx	What commits together
T1	Initialisation: roots in the secret store, then the store header with an `INITIALIZING` marker, then `ACTIVE` only after the pending certificate is durable.
T2	Exact device certificate and vault bytes prepared for activation.
T3	Verified circle or device change, `desiredRevision` increment, active SK invalidation, cancellation of obsolete pending records.
T4	Exact manifest, wraps, SK and audience checkpoint for a prepared epoch.
T5	Epoch publication outcome and conditional promotion with `nextCounter = 0`.
T6	Outbound record into the outbox and local history, counter increment.
T6h	Handoff: eligibility re-check, in-flight mark and the send call for one outbound record.
T7	Inbound record, disposition, conditional live pin, and the acknowledgement cursor under the rule in section 10.
T8	Owner snapshot: the successor revision and its hash, built from local state plus one change.
T9	A standalone vault update before publication, or an accepted reset-key list or receipt.

Required crash outcomes

Before T4 commits, no epoch is publishable; after T4 but before the network succeeds, restart publishes exactly the saved bytes; after remote commit but before T5, lookup finishes T5 without new randomness. Before T6 commits, no `recordId` is returned and nothing is sent; after T6, restart retries the exact wire or cancels it with a disposition, never decrementing the counter; a row marked in flight by T6h whose response was lost is resolved by lookup, never resent under a changed audience. After T7, restart reads history, the pin and the cursor even if callbacks were lost. Before activation commits, the new device stays `activationPending` and emits nothing; after remote activation but before local acknowledgement, lookup finishes that activation and never allocates another device. These outcomes are tested against the real store, not only an in-memory model.

12. Last-resort account reset and the reset keys

Normal device replacement keeps AK, membership and ownership. `resetAccount` is a separate destructive action taken only after both recovery paths have failed, after fresh host re-authentication and an explicit explanation: new account and recovery keys, new device keys, peers see `ACCOUNT_KEY_CHANGED`, membership continuity is lost, re-admission is required, and circles owned by the old identity close. The immutable `accountName` stays. A failed background key lookup never invokes reset.

The old AK may be gone, so it cannot sign its own replacement or its circles' closure. A **host reset authority** signs the receipt instead, with operational keys that a management key can retire. The SDK pins one deployment-specific Ed25519 management public key and a minimum list version; the management private key stays offline, separate from the online signer. The authority can retire identities and close circles; it can never admit a member or decrypt a record. An attacker holding an allowed operational key can deny service until that key is removed from the list.

Type 8: ResetKeyList

```
body = u8(5) || u8(8) || version:u64 || issuedAt:u64 || expiresAt:u64 || keyCount:u8 ||
       rows
row  = authorityKeyId[16] || Ed25519PublicKey[32]
wire = body || Ed25519(managementKey, C("reset-keys-sig") || body)
```

Rows sort strictly by key id; ids and keys are unique, ids nonzero, keys valid points, count 0 to 8. Version starts at 1. `issuedAt < expiresAt` and the lifetime is at most 30 days; a profile renews well inside that. An empty list disables every new reset. Operational keys MUST differ from the management key and cannot sign lists. A retired id is never reused and a compromised key is never re-authorised.

`getResetKeyList()` returns opaque bytes. Verify under the pinned management key; require version at least the packaged minimum and at least the durable highest version. Equal version with equal body hash is a duplicate; equal version with a different valid body is `EQUIVOCATION`; lower is `ROLLBACK`. A higher list fully replaces the allowed set. Persist the accepted bytes, version, hash and highest checked time atomically (T9). Freshness requires `issuedAt <= trustedNow < expiresAt`, where `trustedNow` is the platform clock under an explicit accurate-clock assumption, checked against a durable time high-water mark and monotonic elapsed time; rollback or an unknown clock gives `RESET_CLOCK_UNTRUSTED`. An expired or missing list gives `RESET_KEYS_REQUIRED`: refresh and defer unseen resets. None of this pauses location sharing or authorises a directory key change. The list is cached until expiry.

Type 7: AccountResetReceipt

```
body = u8(5) || u8(7) || B(accountName) || oldIdentityId[32] || newIdentityId[32] ||
       resetId[16] || issuedAt:u64 || authorityKeyId[16]
wire = body || Ed25519(resetKey, C("reset-sig") || body)
```

Old and new ids must differ. For an unseen receipt the SDK requires a fresh key list, an allowed `authorityKeyId`, a valid signature under that key, the account name binding, an old identity that matches its current pin, and the new identity derived from a verified new AK-signed certificate (`RESET_BINDING` otherwise). `issuedAt` is diagnostic: an unseen receipt under a key the current list no longer allows is `RESET_KEY_REVOKED` whatever its date. Already committed resets stay committed; the SDK retains the accepted receipt hash and the immutable (`resetId`, `accountName`, old, new) binding, a duplicate is a no-op, and a conflicting binding for a known `resetId` is rejected. The host may re-attest a legitimate old event under a current key with the same binding.

Host. `resetAccount(expectedOldIdentityId, newCertificate, newVault, resetId, reauthProof)` registers the new identity and device, stores the vault and the receipt, fences old writes, and tombstones every circle owned by the old identity, atomically. The reset vault starts with zero circles and a fresh `recoveryKey`, saved through the recovery provider before the reset is reported complete. Repeated `resetId` returns the same outcome. **Peers.** On a validated receipt each peer records `ACCOUNT_KEY_CHANGED`, stops sharing to the old identity, rotates affected epochs, and treats circles owned by the old identity as closed. The receipt itself is the closure proof. A bare directory key change without a receipt is quarantined, never accepted and never used to close anything.

13. Client contract

`open({ accountName, retentionPolicy, recoveryProvider, resetTrust, secretStore, store, transport, events })` loads protected local state and returns one asynchronous client. `retentionPolicy` is the profile's versioned policy: `deliveryWindowMs`, `appendRetryWindowMs`, `locationHistoryMs`, `checkInHistoryMs`, `hygieneRotationMs`, `maxOutboxRecords`, `maxOutboxBytes`; a missing policy is rejected and the SDK never substitutes defaults. `resetTrust` carries the pinned management key and minimum list version. Public operations return typed results, never plaintext fallbacks. Randomness is internal in production. Adapters pass opaque bytes through unchanged. The host owns login, notifications, positioning, OS permissions and the interface; no plaintext location ever enters the directory or relay API.

Public surface

- `initializeAccount()` creates the account, its first device and vault and saves the `recoveryKey` through the provider; it returns without a recovery code. `recoveryStatus()` reports the provider's `backupState`; `recoveryCode()` reveals the code on demand after local authentication.
- `restoreAccount(optionalCode)` recovers the account and activates a fresh device under it, trying the provider first.
- `enqueueLocation(location)`, `enqueueCheckIn(location | bytes)`, `enqueue(bytes)`: durable outbound records.
- `sync()` verifies metadata first (certificates, snapshots, the reset-key list when due), resolves uncertain activation and control writes, reconciles epochs, drains the outbox, reads and acknowledges batches. Concurrent callers coalesce.
- `receive(wire)` for a record pushed by the host outside `sync`; `history(query)` and `latest(identityId)` read the durable state; `inspect()` returns public metadata only.
- `circles.create()`, `requestJoin(circleId, ownerIdentityId, { shareOnApproval })`, `approve(circleId, accountName)`, `remove(circleId, memberIdentityId)`, `leave(circleId)`, `setEnabled(circleId, bool)`, `closeCircle(circleId)`. Device replacement never calls any of these.
- `resetAccount({ confirmed: true })`. `subscribe(listener)` for state and events. `close()` cancels work and zeroes temporary secrets best-effort.

Transport contract (implemented by the host)

- Directory: `resolveAccounts(names)`, `getCertificate(hash)`, `activateDevice(expectedCertificateHash, expectedVaultRevision, expectedVaultHash, newCertificate, newVault)`, `getVault(accountName)`, `publishVault(expectedRevision, expectedHash, newVault)`, `getCircle(circleId)`, `publishOwnedCircle(circleId, expectedRevision, expectedHash, newSnapshot)`, `requestJoin(circleId)`, `requestLeave(circleId)`, `readRequests(circleId)`, `getResetKeyList()`, `resetAccount(...)`, `getResetReceipt(resetId)`.
- Relay: `getCurrentEpoch(streamId)`, `getEpoch(streamId, epoch)` returning the manifest plus the caller's wrap, `publishEpoch(streamId, expectedEpoch, expectedHash, manifest, wrapMap)`, `appendRecord(recordId, wire)`, `readRecords(streamId, afterSeq, limit)`, `ackThrough(streamId, cursor)`. The host creates a reader's subscription to a stream at admission, at device activation and when an author opens a new stream; the client never asks for one. The transport's `send` MUST be callable from inside T6h without awaiting.

- Notifications are the host's: a visible notification to the owner on a join request and to each recipient on a check-in append; a silent wake-up otherwise. The SDK never composes notification text.
- `observe(handler)` emits `online`, `offline`, `resume` and `metadataChanged` hints. Hints never authorise a state change.

Mutation outcomes are `COMMITTED`, `ALREADY_COMMITTED` (same bytes), `CONFLICT` (with the current bytes), `IGNORED_OLDER`, `RETRYABLE`, `UNAUTHORIZED` or `PERMANENT_FAILURE`. Timeouts are uncertain: re-read the target and compare bytes. Use a 30-second timeout and a uniform random delay of 0 to min(60 s, 2^min(attempt, 6) s) between attempts, one pending operation per target. A retry re-enters T6h like a first send.

14. States, errors and logs

Visible states: `ready`, `locked`, `accountRecoveryRequired`, `activationPending`, `retired`, `paused(sync | rotation | metadata | limit)`, `quarantined`. `ready` means local encryption state is usable, not that the server disclosed globally fresh membership. States are emitted only after the corresponding durable transition. A device change is not a member-left or member-joined event.

Error families. Parsing: `SIZE_LIMIT`, `MALFORMED`, `UNSUPPORTED_VERSION`, `UNSUPPORTED_TYPE`, `BAD_PAYLOAD`. Authentication: `INVALID_KEY`, `BAD_SIGNATURE`, `DECRYPT_FAILED`, `WRAP_HASH_MISMATCH`, `KEY_COMMITMENT_MISMATCH`. Trust and state: `ACCOUNT_KEY_CHANGED`, `ACCOUNT_RECOVERY_REQUIRED`, `DEVICE_ROLLBACK`, `DEVICE_ID_REUSED`, `SUPERSEDED_DEVICE`, `VAULT_BINDING`, `RESET_BINDING`, `RESET_KEYS_REQUIRED`, `RESET_KEY_REVOKED`, `RESET_CLOCK_UNTRUSTED`, `ROLLBACK`, `EQUIVOCATION`, `METADATA_REQUIRED`, `KEY_PENDING`, `NOT_AUTHORIZED`, `ROTATION_REQUIRED`, `COUNTER_EXHAUSTED`, `RECIPIENT_LIMIT`. Delivery: `CLOCK_SKEW`, `EXPIRED`, `QUEUE_FULL`, `CURSOR_OUT_OF_RANGE`, `STALE_APPEND`. Storage and transport: `LOCKED`, `RECOVERY_REQUIRED`, `STORAGE_FAILURE` plus the mutation outcomes. Each error carries a retryable flag and a safe stage name. A wrong recovery secret fails authentication and never triggers a reset.

Safe logs. Allowed fields: a random operation id, stage and result code, recovery provider status and reason, reset-key list version and expiry, device generation, circle and vault revision, epoch and counter, counts, byte lengths, duration and retry count. Never log locations, plaintext, seeds, SK, recovery codes, store keys, session tokens or vault plaintext. Account, circle and record ids are identifying metadata: keep them local or pseudonymise per session. Ciphertext and diagnostic exports are never uploaded automatically. There is no flag that disables verification.

15. Security properties and limits

What is protected. Payload confidentiality against the server and against non-members, under pinned keys: the server never holds `recoveryKey`, `accountSeed`, `deviceSeed` or any SK. Record authenticity and author binding through the DSK signature and the certificate chain to the pinned AK. Membership authenticity through owner signatures over complete snapshots. Continuity across device replacement through stable AK pinning and generation high-water marks, with the replaced device cut off as each author rotates. Rollback and equivocation detection for certificates, snapshots, epochs, records and reset-key lists. Account login alone cannot forge an AK-authorised replacement device. Holding the vault and its recovery key authorises a new device, and with it future access; it decrypts no history.

Attacker	Can	Cannot
Honest-but-curious server	See who shares with whom, when, how often, record sizes, kinds and `sentAt`; infer approximate location from network metadata	Read coordinates, capture times, statuses or check-in content
Malicious server	Substitute a key at an account's first contact; withhold removals, snapshots, certificates, vaults, key lists or records indefinitely; replay an old vault to a recovering device or an unexpired key list for its remaining lifetime; fork views between members; show an owner a join request the named account never made	Produce a document that verifies under a pinned key; roll back a generation, revision, epoch or list version a client has seen; decrypt anything
Removed member	Keep the plaintext and keys it already holds	Open any epoch published after the author learned of the removal
Former device of a member	Keep its own old history; keep opening the epochs it already holds keys for until each author rotates to the new device	Open any wrap addressed to the replacement device; publish as the account after the new certificate is known
Holder of a recovery code or `accountSeed`	Authorise a device of their own and act as the account, visibly, through a new certificate	Read anything without publishing that certificate; this is root compromise, and only reset ends it
Holder of an allowed reset key	Retire identities and close their circles until the key is removed from the list	Admit a member, decrypt a record, or substitute a decrypting key
Another member	Copy and export what it legitimately received	Forge records as someone else

Limits, stated plainly

1. Enrollment is host-trusted, and so are join requests: the owner admits the account a request names under the key the directory serves. The guarantee is continuity of an accepted identity while local trust state survives, subject to the declared recovery paths.
2. Removal takes effect per author, when that author learns of it and rotates. A withholding server delays it without bound; the client keeps sharing with the last verified snapshot and shows its age.
3. Replacement revokes the old device the same way, per author and on each author's next sync. Until an author rotates, an unlocked old device can still open that author's current epoch.
4. No forward secrecy and no post-compromise recovery. Independent epoch keys limit one key's exposure to its epoch; a compromised current device exposes what it holds.
5. Recovery trusts the server's latest vault. An existing device detects a rolled-back vault from its checkpoints; a fresh device cannot. The vault holds no decryption key, so a rolled-back vault costs circle pins and preferences, not plaintext; a restored old sharing preference re-enables sharing only to members the owner admitted.
6. The reset authority can deny service until a key is removed from the list; a client that never syncs never learns the removal. Compromise of the management key requires an app update.
7. A whole-device backup restored on a device with no surviving freshness anchor cannot always be recognised as old. This is not anti-rollback protection.
8. Metadata, authorised-device leaks, positioning truth and the screens of members are outside payload confidentiality.

9. Deletion on the server is a conforming-host policy the client cannot verify; the client's own deletion is what it controls.
10. Delivery and notifications are availability properties, not cryptographic ones. A withholding server can delay a check-in; it cannot forge one.

Audit status. Primitives are libsodium's and audited; the composition is not yet independently reviewed. The vectors in section 16 are implementation evidence, not an audit.

16. Conformance

The companion package is the SS5 revision 2 `vectors.json` plus `verify_vectors.py`: a reproducible reference codec and small state model, not a production SDK. The Kavach v1 fixture in `spec/` is regenerated once from it, by a checker derived from the same source, with four changes: the maximum key-list lifetime is 30 days instead of 24 hours, so case K19 is re-derived at 30 days plus one millisecond; record flag bit 0 is reserved rather than meaning urgent, which leaves the core record untouched and re-derives only the maximum-size record, whose flags move from 1 to 2 (a check-in carrying an opaque 4,096-byte payload), with a new negative vector for bit 0; type 5 is retired, so its document vector is dropped and its wire joins the negative set as `UNSUPPORTED_TYPE`; and the vault entry lost its snapshot pins, so the vault vectors are re-derived with the 49-byte entry and the transfer token is gone. Every other section applies byte for byte, including the two core sealed boxes, the 100-account set, the phone replacement set and the reset-key management cases other than K19. The storage-row fixture is informative only.

What the fixture contains. Five core signed documents spanning types 1 to 4 and 7; two sealed key boxes with their wrap hashes; a 22-byte location plaintext with its AAD, ciphertext, signature and record id; the 100-account boundary set (snapshot 3,324 bytes, manifest 9,808 bytes, wraps 8,000 bytes, maximum record 4,255 bytes); device replacement (generation 2 certificate, unchanged snapshot, continued owner snapshot, new manifest and record, the old device failing to open the new wrap, superseded manifest detection); vault encryption and wrong-key rejection; durable check-ins surviving a later location; an account reset receipt; three key lists with 22 reset-key management cases; 21 negative byte vectors; 3 wrap and key checks; 17 state-model transitions. Ed25519 and X25519 outputs are cross-checked with OpenSSL, XChaCha20-Poly1305 with PyCryptodome; sealed boxes use libsodium on both sides and are not an independent cross-check.

Required tests beyond the checker

- Run the fixture on every supported platform; compare complete bytes, not round trips. Fuzz bounded parsers with a 16 KiB limit. Exercise every boundary: names 1/64/65, payloads 0/4096/4097, counts 0/1/100/101, integer maxima, duplicate and unsorted rows, reserved flags, non-canonical signatures, invalid points, wrong key commitments, vault binding, key-list rollback and expiry.
- Crash on both sides of T1 to T9 and T6h and at every lost-response point, against the real store. Race execution contexts. Reopen and assert no reused counter, no reconstructed nonce, no stale-audience send, exact-byte retry.
- The handoff race: an authorization change committed by one context while another is between eligibility check and send never results in an upload of obsolete ciphertext, tested with the real transport adapter.
- Lost device, two tests. Device keys only: the attacker holds `deviceSeed` and the store but not the account root; after the new device activates and every author has rotated, it opens nothing new. Full state: the attacker also holds `accountSeed` or `recoveryKey` with the vault; the test shows it can still authorise a device of its own, so replacement is not a repair and only reset ends it.

- Acknowledgement: a `keyPending` record never advances the prefix while later records still flow; the relay never deletes a record while a required reader's cursor is below it; an equal or older cursor causes no write; a cursor above the tail is rejected; a reader is returned only records addressed to its certificate.
- Relay: deliver counters 7 then 6 and keep the pin at 7; the record disappears when the last required reader's cursor passes it and at the window without one; expiry advances as a prefix and never expires a newer record; a blocked stream never delays another stream; a retry inside the window returns the original outcome after deletion; an append below the checkpoint without evidence is `STALE_APPEND` and the client marks it `acceptanceUnknown` without re-creating a check-in; a new device's subscription starts at activation and excludes records sealed to the old certificate; every member gets a subscription when an author opens a new stream; epoch blobs disappear only when no record references them and the epoch is not current; a check-in survives a later location end to end.
- Reset: the 22 key-management cases against the real clock adapter, plus wrong key, revoked key with a backdated receipt, binding mismatch, duplicate, conflicting binding, receipt without a verified certificate, expired list deferring a reset without pausing sharing, and no closure on a bare directory key change.

An independent review of the composed protocol, the recovery flows, the store binding, the handoff serialisation and the host's atomicity contract precedes production. Passing the checker is not that review.

17. Questions for the reviewer

1. Is the stable identity (`accountName`, AK) plus a per-device certificate the right unit, or should readers also pin the device id to detect a host that swaps devices between legitimate generations?
2. The manifest binds each recipient's certificate hash. Is anything lost by not binding the recipient's device generation as well?
3. Host-trusted enrollment with optional fingerprint comparison, and host-attested join requests with no requester signature: acceptable for v1, or should an authenticated exchange be required for owners at least?
4. Vault recovery under a random 256-bit key with no password, and no old-device transfer path: is the lack of a vault freshness anchor on a fresh device acceptable given that the vault grants account control and future access but decrypts no history?
5. Reset-key lists with a 30-day maximum lifetime: is the window between a key compromise and the next list a tolerable denial-of-service exposure, and is the trusted-clock assumption acceptable on phones?
6. Deletion when the last required reader's cursor passes a record, with one expiry prefix per stream: any attack the cumulative cursors or the acceptance-time ordering expose?
7. The acknowledgement rule holds the prefix at a `keyPending` record while newer records are stored, and the relay filters by subscription. Is there a sequence where a reader acknowledges and still lacks plaintext it needs?
8. The live pin advances on (generation, epoch, counter) while history accepts out-of-order counters. Is there a replay that moves the pin forward illegitimately?
9. A cross-process lock with the send call initiated inside it as the only serialisation of the handoff: sufficient on both mobile platforms' background execution models, and is a send that fails synchronously handled safely?
10. Check-ins share the record counter and differ by one authenticated bit. Should they have their own stream instead?

11. What should the first external audit cover beyond the composition: the recovery flows, the host atomicity, or the platform adapters?

Appendix. Scenario tests that gate release

Each scenario is an executable test against the real client with a fake host; the relay scenarios also run against the host's real relay. They gate the delivery and replacement milestones of the build sequence.

1. Create a circle, request a join, approve it: the owner is notified once; the requester's first manifest names both devices; the owner's audience rotates once.
2. Approve a join while the requester is offline: on its return the requester enables sharing from its vault preference without a second approval.
3. Owner replaces device with a member offline: the circle revision is unchanged; the owner signs revision plus one afterwards; the member rotates to the new device on return.
4. Member replaces device while the owner is offline: each online author rotates to the new `DEK` within one sync without owner involvement; the old device cannot open the new wraps.
5. Lost device with full old-device state in the attacker's hands: after activation and every author's rotation the attacker opens nothing new; the test records what it could open before the last rotation.
6. Two candidate devices activate concurrently: exactly one commits; the other reads the committed state and reports it.
7. Lose the local store, keep recovery: membership survives; the new stream starts at epoch 1, counter 0; no old counter is reused.
8. Enrollment with the recovery provider unavailable: the account is created, sharing works, recovery status reports not backed up, and the code is reachable later.
9. Platform recovery to a second device with no old device and no code; a wrong platform account yields `unavailable`, never a new account.
10. Both recovery paths fail: `ACCOUNT_RECOVERY_REQUIRED`; circles remain intact; nothing closes.
11. Reset with a valid receipt under an allowed key: peers close owned circles and re-admit the new identity only after owner approval; the same receipt under a key removed by a newer list is `RESET_KEY_REVOKED`; a bare directory key change closes nothing; an expired list defers the reset and does not pause sharing.
12. Removal with the owner online: the author pauses, rotates and the removed member cannot open the next epoch.
13. Removal withheld by the server: the author keeps sharing with the last verified snapshot and shows its age.
14. Remove then rejoin between two syncs: `desiredRevision` still increments and the epoch rotates.
15. Over-limit union across circles: `RECIPIENT_LIMIT`, nothing truncated.
16. Crash after `T4` and before publish: restart publishes identical bytes. Crash after remote commit and before `T5`: lookup promotes without new randomness.
17. Membership changes while an epoch is in flight: the epoch commits obsolete; enqueue returns `ROTATION_REQUIRED`; the next epoch promotes.
18. Crash after `T6` and before send: restart retries the exact wire; crash after a membership change cancels it with a disposition.

19. Handoff race: a membership change committed by a background context while the foreground sender is between its check and its send never produces an upload sealed for the old audience.
20. Deliver counters 7 then 6: both in history, pin stays at 7.
21. Record arrives while offline before its epoch could be fetched: `keyPending`, the acknowledged prefix holds, later records are stored, the epoch is fetched on the next sync and the record is read again and opened; the relay still held it.
22. Last recipient acknowledges: the record is gone from the relay; the epoch blobs are gone once the epoch is not current and nothing references it.
23. A recipient stays away past the window: the record is deleted unacknowledged; the author's history still has it.
24. Check-in, then two locations: three records delivered; the check-in survives on relay and reader; the pin is the last location; recipients were notified of the check-in visibly and of the locations silently.
25. Two execution contexts enqueue concurrently: counters are unique and the outbox order is preserved.
26. Hygiene rotation at the configured interval and counter exhaustion: both produce a new epoch and `needsLocation`.
27. Superseded device sends after its replacement activated: the relay refuses; a reader that receives it anyway stores history only and keeps the pin.